

COMP 581: Introduction to Robotics

Spring 2026

Instructor: Daniel Szafir (daniel.szafir@cs.unc.edu)

Teaching Assistant: Thayer Hicks (thayerh@cs.unc.edu)

Lab 4: Bug Race

Deadline

- Lab pre-flection due **Friday, April 10 at 11:59pm**. To pass this part of the lab, please submit your answers in a group submission on GradeScope (see the Lab 4 Pre-flection assignment link on Canvas).
- Lab simulation: No due date. Getting the Webots simulation running correctly is optional for this lab and meant to be a useful aid for you in getting your real robot working properly.
- Robot lab due **Monday, April 27 at the beginning of class (11:15 am)**. Before the due time, each team should submit their final code by committing and pushing to GitHub; see specific instructions below. For the late policy, see the course syllabus.
- Lab reflection due **Monday, April 27 at 11:59pm**. To pass this part of the lab, please submit your answers in a group submission on GradeScope (see the Lab 4 Reflection assignment link on Canvas).

The Objective

You will build and program a robot capable of moving as quickly as possible from a start location to a known goal location in an unknown environment while avoiding static obstacles.

Robot Design Requirements

Your robot must be built only using pieces from your Lego Mindstorms kit. You should design your robot such that the dark gray center button may be pressed without moving the robot and such that pushing the button does not disrupt any of the sensors. Your robot should remain in one piece at all times, its diameter with respect to the ground should not exceed 40 cm, and its diameter shall not change substantially during the task. All measurements will be made with respect to the **measuring point**: a distinct point of your choosing that is on the **frontmost surface** of your robot. More specifically, you will place your robot on the ground behind a starting line drawn on the ground, such that your chosen measuring point is **directly above a point on the starting line marked by a star**, and no part of your robot is permitted to be in front of the starting line. Your robot should be designed to accomplish the task autonomously without any human intervention (except for pushing the dark gray center button as required).

Setup

We have created a Webots simulation library that will enable you to write EV3 MicroPython compatible code in Webots to control a simulated Lego robot in a world that mimics the Lab 4 environment. To download everything you need for this simulation and for deploying your program to the real robot:

1. Log into your GitHub account and then accept the GitHub Classroom assignment by clicking this link: <https://classroom.github.com/a/rXLLujPm>
2. The link will take you to a page where you can accept the assignment on GitHub.
3. After accepting the assignment, it should create the repository for you and your team partner automatically based on the teams created in Lab 1. If you have any difficulty with this, please let us know.
4. The repository will be created and set up in a similar manner as in Labs 1–3. Please view the repository README for some additional details specific to this lab. You should now be able to edit the `main.py` controller file within the controllers folder in Visual Studio Code and see the updates to the robot's behavior when you run the Webots simulation.
 - Always remember to reset the Webots time back to zero.
5. Your job is now to write the `main.py` controller to solve the challenge described in the next section below and deploy your solution on your physical robot.

The Task

In this lab, your robot will race to a predefined goal location as fast as possible, while being able to autonomously navigate around any obstacles that might lie between the start and goal positions. The task in this lab is to be completed in a single run, without picking up your robot.

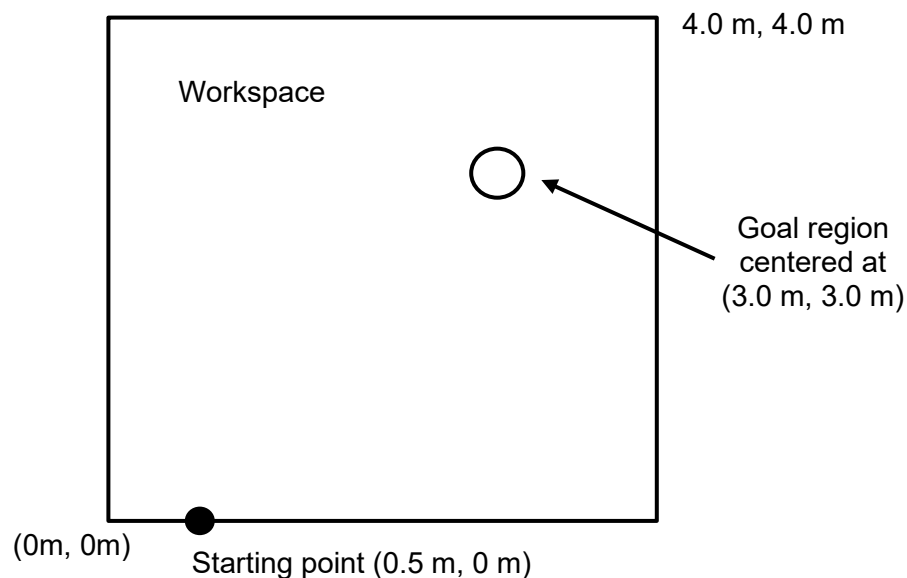
Pre-flection:

Before programming anything, first read through the task below and make sure you understand it. Then, work with your lab partner (or complete separately and then compare answers) to fill in the Lab 4 Pre-flection worksheet linked in the Lab 4 Pre-flection assignment on Canvas and submit this through GradeScope (only one submission needed per group).

Race Task:

You will place your robot behind a starting line such that its measuring point will be on a specific *starting point* (50 cm, 0 cm) on the starting line. The *goal point* is located at (300 cm, 300 cm) and the *goal region* is defined as the region that is within 5 cm of the goal point. All robot motion must be completed within the 400 cm x 400 cm workspace, with the sole exception that the robot may exit the workspace if it remains within 30 cm of an obstacle.

The objective for your robot is to autonomously move from the starting point to the goal region as fast as possible while staying in the workspace. Upon pushing the center button, your robot should begin the task. When reaching the goal region, the robot should stop and beep. The figure below illustrates the workspace, starting location, and goal region (note that this workspace will also contain obstacles, which are not shown):



There will be between 2 and 6 obstacles in the workspace. Your robot will not know the shapes or locations of the obstacles before starting the task. Each obstacle will consist of bumpable, ultrasonic reflective walls that are at least 17 cm tall. Each obstacle will be a closed shape made of straight and curved segments and may contain corners. Shapes are not necessarily convex. No gap on the ground between two obstacles will be less than 50 cm wide.

The task must be completed in under 4 minutes.

Grading Rubric

- **(5 Points) Pre-flection:**
 - Effort: 5 points. Points will be awarded for submitting the Pre-flection on time and for making a reasonable effort to answer each question.
- **(90 Points) Real Robot Attempt:**
 - Effort: 10 points. You will receive these points by:
 - Showing up to the lab on time: 6 points
 - Having your robot built and ready to attempt the course: 2 point
 - Pointing out your robot's measuring point according to the definition given in this document: 2 point
 - Robot Task: 80 points, with a breakdown as follows:
 - Moving around obstacles: 50 points. If your robot stops in the goal region, you will receive full credit for moving around obstacles. If your robot does not stop in the goal region, some points will be awarded if your robot follows the wall of at least one obstacle it encounters, based on the number of edges followed.
 - Goal accuracy: 20 points. Points will be awarded based on proximity to the goal zone when your robot stops. If your robot does not stop autonomously before the time limit, no goal accuracy points will be awarded.
 - Time: 10 points. Points will be awarded based on time elapsed from when your robot's dark gray center button is pressed at the beginning until the robot stops. Faster completion will be awarded more points. If your robot does not stop in the goal region, no points for time will be awarded.
- **(5 Points) Reflection:**
 - Effort: 5 points. Points will be awarded for submitting the Reflection on time and for making a reasonable effort to answer each question.

Extra Credit Opportunities:

- Style: Up to 5 bonus points for interesting things you do with your robots. This can be anything from sound effects to additional Lego parts/colors, have fun!

Penalties:

- There will be a -10 point penalty if, after crossing the start line, your robot exits the feasible workspace (unless it is doing so while remaining 30cm from an obstacle).
- There will be a -10 point penalty if more than 4 minutes elapse from when the dark gray center button is pushed at the start location until the robot stops. If the robot is still moving after 4:30 minutes, then the instructor can stop the run and no additional points can be earned.

Code

To complete this lab, you will use a subset of the EV3 MicroPython API. You may only use the following modules:

[ev3brick](#)

[ev3devices](#)

[parameters](#)

[tools](#)

[media](#)

You may *not* import, use, or copy the “robotics” module, or any of its included classes or functions, in your program.

A robot relying on a module not allowed as stated above will be disqualified. Also, any robot using Wi-Fi or Bluetooth communications in any way will be disqualified.

You should submit a zip file of your Python source code files on GradeScope. The zip file should include all the Python source code files you wrote (which should have a .py file extension). In the comments at the top of the Python source file containing your main .py file, please include the names and PIDs of all team members.

You are free to discuss concepts in the course with other students or AI tools (e.g., ChatGPT) in natural language, but you should not share any programming code with others and you must write all the robot’s programming code yourself. If you access any sources other than the textbook or course material posted on Canvas, you must cite them in comments at the top of your code. You must fully understand your code and be able to reproduce the algorithmic approach without references if asked. The Honor Code is in effect for these policies.

Good Luck and Have Fun!